WEEK NO:06

DATE:09 January,2026

SUBJECT: Python Programming Lab

1.Question

Write a python program to work with string built-in functions.

Procedure:

- Take a string input from the user.
- Apply different built-in string functions like upper(), lower(), title(), strip(), find(), replace(), split().
- Display results**.**

Program:

```
s = input("Enter a string: ")

print("Original String:", s)
print("Upper case:", s.upper())
print("Lower case:", s.lower())
print("Capitalize:",s.capitalize())
print("Title case:", s.title())
print("Length:", len(s))
print("Find 'a':", s.find('a'))
print("Replace a with x:", s.replace('a','x'))
print("Split words:", s.split())
```

Output:

```
Enter a string: Divya sree
Original String: Divya sree
Upper case: DIVYA SREE
Lower case: divya sree
Capitalize: Divya sree
Title case: Divya Sree
Length: 10
Find 'a': 4
Replace a with x: Divyx sree
Split words: ['Divya', 'sree']
```

Observations:

- If you misspell any function name or brackets the it give error like this AttributeError: 'str' object has no attribute 'captalize'.
- find() returns **-1** if the character is missing.
- Python strings are **case-sensitive.**

- Strings are immutable.

2.Question:

Write a python program to determine n u m b e r o f times a given letter occurs in a string.

Procedure:

- Read a string and a letter from user.
- Use count() function to find occurrences.
- Print result.

Program:

```
s = input("Enter a string: ")

ch = input("Enter a character to count: ")

 count = s.count(ch)

print("Number of times", ch, "occurs:", count)
```

Output:

Enter a string: Divya sree

Enter a character to count: e

Number of times e occurs: 2

Observations:

- We get logical error if we enter more than one character.
- If user enters space as character it counts spaces also.
- Python is case-sensitive. If we enter ‘a’ instead of ‘A’ -> they both are different.

3.Question:

Write a python program to check if a string is a palindrome or not.

Procedure:

- Take a string input.
- Reverse it using slicing [::-1].
- Compare original and reversed strings.

Program:

```
s = input("Enter a string: ")

rev = s[::-1]
```

```
if s == rev:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Output:

Enter a string: mam

Palindrome

Observations:

- A palindrome reads same forward and backward.
- Special symbols like '!' affect comparision.
- s[::-1] is **string slicing** used to reverse a string.

4.Question:

Illustrate in operator and write a python program to count number of lowercase characters in a string.

Procedure:

- Take a string input.
- Check if a specific letter exists using in.
- Use loop and islower() to count lowercase letters.

Program:

```
s = input("Enter a string: ")
if 'a' in s:
    print("'a' is present in the string")
else:
    print("'a' is not present")
count = 0
for ch in s:
    if ch.islower():
    count += 1
```

```
print("Number of lowercase letters:", count)
```

Output:
Enter a string: Divya
'a' is present in the string
Number of lowercase letters: 4

Observations:

- ‘in’ operator checks membership.
- islower() counts only alphabetic lowercase letters, numbers and symbols are ignored.
- count variable stores total lowercase letters.

5.Question:

Write a program to replace all the occurrences of letter ‘a’ with letter ‘x’ in a string.

Procedure:

- Read a string input.
- Use replace() function.
- Display modified string.

Program:

```
s = input("Enter a string: ")

new_str = s.replace('a','x')

print("Modified String:", new_str)
```

Output:

Enter a string: Divya

Modified String: Divyx

Observations:

- replace(old,new) replaces all occurrences.
- Replaces only lowercase `'a'` ,Uppercase `'A'` is not changed.
- If 'a' is not in the string, output remains same.

WEEK NO: 07

DATE: 13 February,2026

SUBJECT: Python Programming Lab

1.Question

Write a program to implement the following list functions a)len() b)extend() c)sort() d) append() e)insert() f)remove().

Procedure:

- Create a list.
- Apply list functions len(), extend(), sort(), append(), insert(), remove().
- Display the results after each operation.

Program:

```
l = [1,2,3,4]

print("Original List:", l)

print("Length of list:", len(l))

l.append(5)

print("After append:", l)

l.insert(2, 10)

print("After insert:", l)

l.extend([6, 7])

print("After extend:", l)

l.remove(4)

print("After remove:", l)

l.sort()

print("After sort:", l)
```

Output:

Original List: [1, 2, 3, 4]

Length of list: 4

After append: [1, 2, 3, 4, 5]

After insert: [1, 2, 10, 3, 4, 5]

After extend: [1, 2, 10, 3, 4, 5, 6, 7]

After remove: [1, 2, 10, 3, 5, 6, 7]

After sort: [1, 2, 3, 5, 6, 7, 10]

Observations:

- append() adds at end, insert(index,number) adds at position.
- If a number is not present in the list, Python gives: ValueError: list.remove(x): x not in list.
- If list contains mixed data types while sorting it gives : TypeError because numbers and strings cannot be compared.

2.Question

Write a program to pass list as an argument to a function.

Procedure:

- Define a function that accepts a list.
- Print elements inside function.
- Call function with a list.

Program:

```
def show_list(a):

    print("Elements in list are:")

    for i in a:

    print(i)

l = [10, 20, 30, 40]

show_list(l)
```

Output:

Elements in list are:

10

20

30

40

Observations:

- proper indentation inside function is mandatory in python otherwise python gives: IdentationError.
- Lists are iterable objects in Python.
- If we modify a inside the function, the original list may change.

3.Question:

Write a python program to find the largest and smallest number in a list.

Procedure:

- Take list input.
- Use max() and min() functions.
- Display results.

Program:

```
l = [1,2,3,4,5]

print("Largest element:", max(l))

print("Smallest element:", min(l))
```

Output:

Largest element: 5

Smallest element: 1

Observations:

- max() returns highest value.
- min() returns lowest value.
- If list contains negative numbers Program still works correctly.

4.Question:

Write a python program to merge two lists and sort it.

Procedure:

- Create two lists.
- Merge using + operator .
- Sort the merged list using sort().

Program:

```
l1 = [5, 1, 9]

l2 = [2, 8, 3]

l3 = l1 + l2

print("After merging List:",l3)

l3.sort()

print("After SortingList:", l3)
```

Output:

After merging List: [5, 1, 9, 2, 8, 3]

After SortingList: [1, 2, 3, 5, 8, 9]

Observations:

- '+' operator combines lists.
- sort() arranges elements in ascending order.
- sort() modifies the list **in-place** (no new list created).

5.Question

Write a python program to remove the duplicate items from a list.

Procedure:

- Take a list with duplicates.
- Use loop to check and build new list without duplicates by using in and not in.

Program:

```
l = [1, 2, 2, 3, 4, 4, 5]

list= set()

new_list = []

for i in l:

        if i not in list:

        new_list.append(i)

        list.add(i)
```

```
print("List after removing duplicates:", new_list)
```

Output:

List after removing duplicates: [1, 2, 3, 4, 5]

Observations:

- not in helps avoid repeated values.
- Lists are **mutable**, so append() changes new_list directly.
- set() is used to store already seen elements.

6.Question

Write a python program to find the sum of elements in a list.

Procedure:

- Create a list.
- Use loop or sum() function.
- Print total.

Program:

```
l = [1,2,3,4,5]

total = 0

for i in l:

        total =total+i

print("Sum of elements:", total)
```

Output:

Sum of elements: 15

Observations:

- If list contains strings: TypeError: unsupported operand type.
- Variable total acts as an accumulator.
- Lists are iterable , so for i in l works directly.

WEEK NO: 08

DATE: 20 February,2026

SUBJECT: Python Programming Lab

1.Question

Write a program to create a list of tuples with the first element as the number and the second element as the square of the first element.

Procedure:

1. Start the program.
2. Use `range(1,6)` to generate numbers from **1 to 5**.
   Apply **list comprehension**:
   - For each number `n` in the range,
   - Create a tuple `(n, n**2)`.
3. Store all tuples in the list named `numbers`.
4. Print the list using `print(numbers)`.
5. Stop the program.

Program:

```
numbers=[(n,n**2) for n in range(1,6)]

print(numbers)
```

Output:

[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

Observations:

- Forgetting square brackets `[ ]` in list comprehension lead to syntaxError.
- Using an undefined variable:NameError.
- If range values are not integers:TypeError(Range only accepts integers.)
- `n**2` calculates the square of each number.

2.Question

Write a python program that takes the list of tuples and sorts the list of tuples in increasing order by the last element in each tuple.

Procedure:

1. Start the program.

2. Create a list containing tuples.

3. Use the `sorted()` function to sort the list.
4. Use `key=lambda x: x[-1]`:
   - `x` represents each tuple.
   - `x[-1]` selects the last element of the tuple.
5. Store the sorted result in `sorted_list`.
6. Print the sorted list.
7. Stop the program.

Program:

```
list=[(10,4),(5,4),(3,7),(8,2)]

sorted_list=sorted(list,key=lambda x:x[-1])

print("Sorted list;",sorted_list)
```

Output:

Sorted list; [(8, 2), (10, 4), (5, 4), (3, 7)]

Observations:

- Missing colon in lambda:Syntax Error.
- If tuples are empty - IndexError: tuple index out of range.
- The sorting is done in **ascending order** by default.
- The `sorted()` function is used to sort the elements of a list.

3.Question

Write a program to implement the following dictionary methods a) keys() b) values() c)items() d) pop() e)delete().

Procedure:

- Start the program.
- Create a dictionary named `student` with key-value pairs.
- Use `keys()` to display all dictionary keys.
- Use `values()` to display all dictionary values.
- Use `items()` to display key-value pairs as tuples.
- Use `pop()` to remove a specific key and store the removed value.
  Use `del` to delete a key-value pair from the dictionary.
  Print the dictionary after each operation.
  Stop the program.

Program:

```
student = {
    "name": "Divya",
    "roll_no": 101,
    "course": "B.Tech",
    "marks": 85
}
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())
removed_value = student.pop("marks")
print("Removed value using pop():", removed_value)
print("Dictionary after pop():", student)
del student["course"]
print("Dictionary after delete:", student)
```

Output:

Keys: dict_keys(['name', 'roll_no', 'course', 'marks'])

Values: dict_values(['Divya', 101, 'B.Tech', 85])

Items: dict_items([('name', 'Divya'), ('roll_no', 101), ('course', 'B.Tech'), ('marks', 85)])

Removed value using pop(): 85

Dictionary after pop(): {'name': 'Divya', 'roll_no': 101, 'course': 'B.Tech'}

Dictionary after delete: {'name': 'Divya', 'roll_no': 101}

Observations:

- If you forgot a comma:SyntaxError.
- `items()` returns key-value pairs in tuple form.
- `del dictionary[key]` permanently deletes the key-value pair.
- `pop(key)` removes the specified key and returns its value.
- After deletion, the dictionary size decreases.

4.Question

Write a python program to add a key value pair to a dictionary and update the dictionary based on the key.

Procedure:

- Start the program.
- Create a dictionary named `student`.
- Print the original dictionary.
- Add a new key-value pair and Print the dictionary after adding a new key.
- Update an existing key by assigning a new value.
- Print the dictionary after updating.
- Stop the program.

Program:

```
student = {
    "name": "Divya",
    "roll_no": 101,
    "marks": 85
}
print("Original Dictionary:", student)
student["course"] = "B.Tech"
print("After Adding New Key:", student)
student["marks"] = 95
print("After Updating Marks:", student)
```

Output:

Original Dictionary: {'name': 'Divya', 'roll_no': 101, 'marks': 85}

After Adding New Key: {'name': 'Divya', 'roll_no': 101, 'marks': 85, 'course': 'B.Tech'}

After Updating Marks: {'name': 'Divya', 'roll_no': 101, 'marks': 95, 'course': 'B.Tech'}

Observations:

- A new key-value pair can be added using `dictionary[key] = value`.

- If the key already exists, the value is updated automatically.
- Dictionaries are mutable (can be modified).
- Keys must be unique in a dictionary.
- If we are trying to access a key that does not exist:**KeyError.**

5.Question

Write a Program to do a reverse dictionary lookup in python.

Procedure:

- Start the program.
- Create a dictionary named `fruits` with key-value pairs.
- Store the value to be searched in `value_to_find`.
- Use `for key, value in fruits.items()` to iterate.
- Compare each value with the search value.
- If matched, print the corresponding key.
- Stop the program.

Program:

```
fruits = {
    "a": "Apple",
    "b": "Banana",
    "c": "Cherry",
    "d": "Mango"
}
value_to_find = "Cherry"
for key, value in fruits.items():
    if value == value_to_find:
        print("Key for value", value_to_find, "is:", key)
        break
else:
    print("Value not found")
```

Output:

Key for value Cherry is: c

Observations:

- Dictionary normally searches using keys, not values but Reverse lookup requires looping through the dictionary.
- If the value exists, its key is printed.
- `items()` returns key-value pairs.
- If multiple keys have same value, only first key is returned (because of `break`).
- Dictionary values are case-sensitive.
- Modifying Dictionary Inside Loop gives RuntimeError: dictionary changed size during iteration.

WEEK NO: 09

DATE: 27 February,2026

SUBJECT: Python Programming Lab

1.Question

Write a program to implement read(), readline(), readlines(), write(), writelines() methods on files.

Procedure:

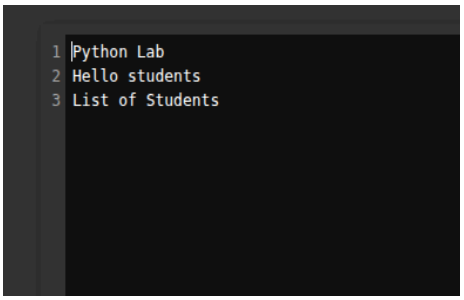
- Start the program.
- Open the file `"Student.txt"` in write mode (`"w+"`).
- Write text into the file using:
  - `write()` method
  - `writelines()` method
- Close the file automatically using `with` statement.
- Open the file in read mode (`"r"`).
- Use `read()` to display entire file content.
- Open the file again and use `readline()` to read one line.
- Open the file again and use `readlines()` to read all lines as a list.
- Display the outputs.
- Stop the program.

Program:

```
with open("Student.txt", "w") as f:

    f.write("Python Lab\n")

    f.write("Hello Students\n")

    f.writelines(["List of ", "Students"])

with open("Student.txt", "r") as f:

    print("Using read():")

    print(f.read())

with open("Student.txt", "r") as f:

    print("Using readline():")

    print(f.readline())
```

```
with open("Student.txt", "r") as f:
    print("Using readlines():")
    print(f.readlines())
```

Output:



Using read():

Python Lab

Hello Students

List of Students

Using readline():

Python Lab

Using readlines():

['Python Lab\n', 'Hello Students\n', 'List of Students']

Observations:

- `writelines()` writes multiple strings (does not automatically add newline).
- `read()` reads the complete file content.
- `readline()` reads only one line at a time.
- `readlines()` returns all lines in list format.
- `with` statement automatically closes the file.

2.Question

Write a program to implement seek(), tell() and flush() methods with different arguments in a file.

Procedure:

- Start the program.

- Open the file in write and read mode.
- Write data into the file.
- Use `tell()` to check the current file pointer position.
- Use `seek()` to move the file pointer to a specific position.
- Read the content from that position.
- Display the final pointer position.
- Stop the program.

Program:

```
with open("Student.txt", "w+") as f:

    f.write("Hello World\nPython File Handling")

    print("Current position:", f.tell())

    f.seek(0)

    print("After seek(0):", f.readline())

    f.seek(6)

    print("After seek(6):", f.readline())

    print("Final position:", f.tell())
```

Output:

Current position: 32

After seek(0): Hello World

After seek(6): World

Final position: 12

Observations:

- `flush()` ensures data is written before reading.
- If file is opened in `"w"` mode and you try to read like:

  open("Student.txt", "w")

f.readline()

then it gives:io.UnsupportedOperation: not readable

- If `seek()` is given invalid position:ValueError
- `tell()` returns the current file pointer position.

3.Question

Write a program to generate 20 random numbers in the range of 1 to100 and write to a file.

Procedure:

- Start the program.
- Import the `random` module.
- Open a file in write (`"w"`) mode.
- Use a loop to generate 20 random numbers using `random.randint(1,100)`.
- Write each number into the file with newline.
- Close the file automatically using `with` statement.
- Display success message.
- Stop the program.

Program:

```
import random

with open("random_numbers.txt", "w") as f:

    for i in range(20):

        num = random.randint(1, 100)

        f.write(str(num) + "\n")


print("20 random numbers generated and written to file.")
```

Output:

20 random numbers generated and written to file.

Observations:

- **ModuleNotFoundError** – If `random` module is not imported.
- `random.randint(1,100)` generates numbers between 1 and 100 .
- **TypeError** – If integer is written without converting to string.
- If `"a"` (append mode) is used instead of `"w"`, new random numbers will be added to the existing file instead of replacing it.

WEEK NO: 10

DATE: 06 March,2026

SUBJECT: Python Programming Lab

1.Question

Program to illustrate how to import data from CSV to DataFrame using Pandas.

Procedure

- Import pandas library
    - Create or use a CSV file
    - Read CSV file using `read_csv()`
- Display DataFrame

Program

```
import pandas as pd

data = {

    "Name": ["Asha", "Ravi", "Kiran"],

    "Age": [20, 21, 19],

    "Marks": [85, 90, 88]

}

df_create = pd.DataFrame(data)

df_create.to_csv("students.csv", index=False)

df = pd.read_csv("students.csv")

print("Data imported from CSV:")

print(df)
```

Output

```
Data imported from CSV:

   Name  Age  Marks

0  Asha   20     85

1  Ravi   21     90
```

2 Kiran 19 88

Observations

- read_csv() is used to load CSV data into a DataFrame
    - DataFrame displays tabular data like a table
- to_csv() helps create CSV files for testing

2.Question

Program to illustrate how to Inspect data in DataFrame using head(), tail () and describe() functions.

Procedure

- Import pandas
    - Create DataFrame
    - Use head() to view first rows
    - Use tail() to view last rows
- Use describe() for statistical summary

Program

```
import pandas as pd

data = {

        "Name": ["A", "B", "C", "D", "E"],

        "Age": [20, 21, 19, 22, 20],

        "Marks": [85, 90, 88, 76, 95]

}

df = pd.DataFrame(data)

print("Using head():")

print(df.head())

print("\nUsing tail():")

print(df.tail())

print("\nUsing describe():")

print(df.describe())
```

Output

Using head():

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	88
3	D	22	76
4	E	20	95

Using tail():

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	88
3	D	22	76
4	E	20	95

Using describe():

	Age	Marks
count	5.000000	5.000000
mean	20.400000	86.800000
std	1.140175	7.155418
min	19.000000	76.000000
25%	20.000000	85.000000
50%	20.000000	88.000000
75%	21.000000	90.000000

```
max     22.000000  95.000000
```

Observations

- head() shows first 5 rows (default)
    - tail() shows last 5 rows
- describe() gives statistical details like mean, min, max

3.Question

Program to perform sorting and slicing operations.

Procedure

- Create DataFrame
    - Sort data using sort_values()
    - Slice rows using indexing
- Display results

Program

```
import pandas as pd

data = {

        "Name": ["Asha", "Ravi", "Kiran", "Meena"],

        "Age": [20, 21, 19, 22],

        "Marks": [85, 90, 88, 76]

}

df = pd.DataFrame(data)

print("Original DataFrame:")

print(df)

sorted_df = df.sort_values(by="Marks")

print("\nSorted DataFrame (by Marks):")

print(sorted_df)

print("\nSlicing first 2 rows:")

print(df[:2])

print("\nSlicing Name and Marks columns:")
```

```
print(df[["Name", "Marks"]])
```

Output

```
Original DataFrame:

     Name  Age  Marks
0   Asha   20    85
1   Ravi   21    90
2  Kiran   19    88
3  Meena   22  76

Sorted DataFrame (by Marks):

     Name  Age  Marks
3  Meena   22  76
0   Asha   20    85
2  Kiran   19    88
1   Ravi   21    90

Slicing first 2 rows:

   Name  Age  Marks
0  Asha   20     85
1  Ravi   21     90

Slicing Name and Marks columns:

    Name     Marks
0   Asha     85
1   Ravi     90
2  Kiran         88
```

```
3  Meena        76
```

Observations

- sort_values() sorts data based on a column
- df[:n] slices first n rows
- Selecting columns is done using df[[col1, col2]]

WEEK NO: 11

DATE:14 March,2026

SUBJECT: Python Programming Lab

1.Question

Design and develop a GUI application to display —Hello World.

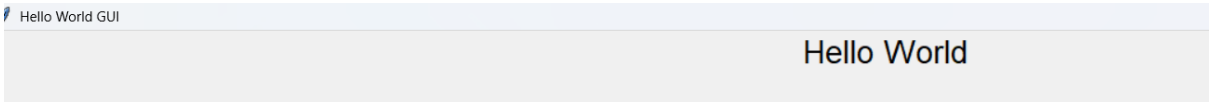
Procedure

1. Import all required classes from the Tkinter module.
2. Create the main application window using `Tk()`.
3. Set the title of the window.
4. Add a `Label` widget with the text “Hello World”.
5. Use `pack()` to place the label in the window.
6. Run the GUI using `mainloop()`.

Program

```
from tkinter import *

root = Tk()

root.title("Hello World GUI")

label = Label(root, text="Hello World", font=("Arial", 20))

label.pack()

root.mainloop()
```

output



Observations

- Runtime Error: occurs if `mainloop()` is not used properly, causing the window to close immediately.
- The GUI window is created successfully using `Tk()`.
- The text **“Hello World”** is displayed using the `Label` widget.
- The `pack()` method places the label in the window.
- The window remains active because of `mainloop()`.

2.Question

Design and develop a GUI application using Label, Entry and Button widgets.

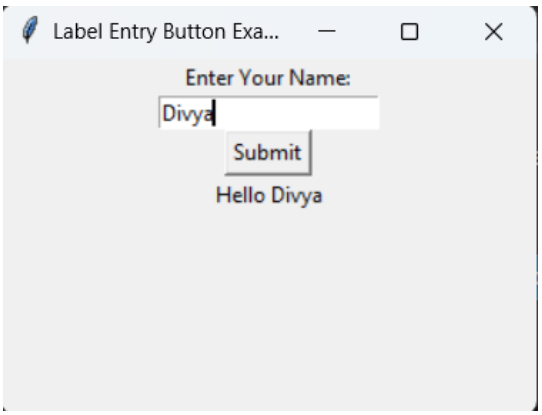
Procedure

- Import the required module from Tkinter.
- Create the main window using `Tk()`.
- Add a `Label` widget to display text.
- Add an `Entry` widget to accept user input.
- Add a `Button` widget to perform an action.
- Use `pack()` to arrange all widgets.
- Run the window using `mainloop()`.

Program

```
from tkinter import *

root = Tk()

root.title("Label Entry Button Demo")

root.geometry("300x200")

def show_text():

    text = entry.get()

    result.config(text="Hello " + text)

label = Label(root, text="Enter Your Name:")

label.pack()

entry = Entry(root)

entry.pack()

Button(root, text="Submit", command=show_text).pack()

result = Label(root, text="")

result.pack()

root.mainloop()
```

output



Observations

- `Entry` widget is used to accept input from the user.
- `entry.get()` reads the entered text.
- `config()` updates the output label dynamically.
- **Runtime Error** → function name not properly linked in button command.
- **TclError** → occurs when invalid widget options are given, such as wrong font format or invalid geometry value.
- **Command Error** → occurs when button `command` is linked to a function name that is not defined.

3.Question

Design and develop a GUI application using Tkinter Geometry methods pack(),grid(), place().

Procedure

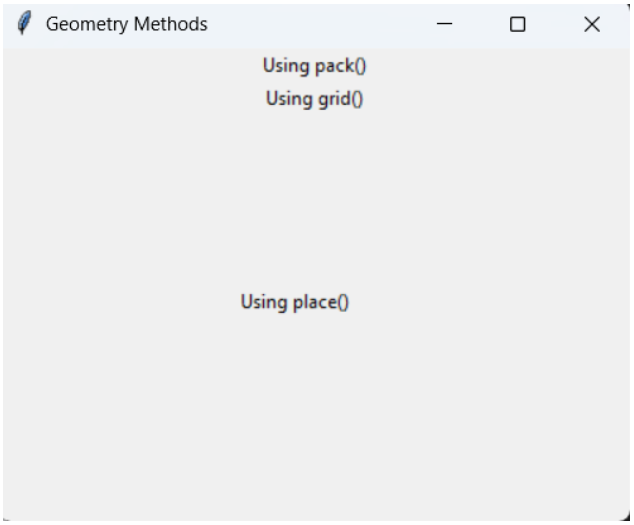
1. Import the required module from Tkinter.
2. Create the main window using `Tk()`.
3. Add widgets such as labels.
4. Arrange one widget using `pack()`.
5. Arrange another widget using `grid()`.
6. Arrange one more widget using `place()`.
7. Run the application using `mainloop()`.

Program

```
from tkinter import *

root = Tk()

root.title("Geometry Methods")

root.geometry("400x300")
```

```
frame1 = Frame(root)

frame1.pack()

label1 = Label(frame1, text="Using pack()")

label1.pack()

frame2 = Frame(root)

frame2.pack()

label2 = Label(frame2, text="Using grid()")

label2.grid(row=0, column=0)

label3 = Label(root, text="Using place()")

label3.place(x=150, y=150)

root.mainloop()
```

Output



Observations

- `pack()` arranges widgets automatically in vertical or horizontal order.
- `grid()` places widgets in **rows and columns**.
- `place()` positions widgets using exact `x` and `y` coordinates.
- Widgets are displayed properly when each geometry method is used in separate containers (frames).
- If `pack()` and `grid()` are used in the same parent window, an error occurs.
- **TclError** → occurs when invalid geometry options are used or when `pack()` and `grid()` are mixed in the same window.

4.Question

Design and develop a GUI application using CheckButton and Radiobuttonwidgets.

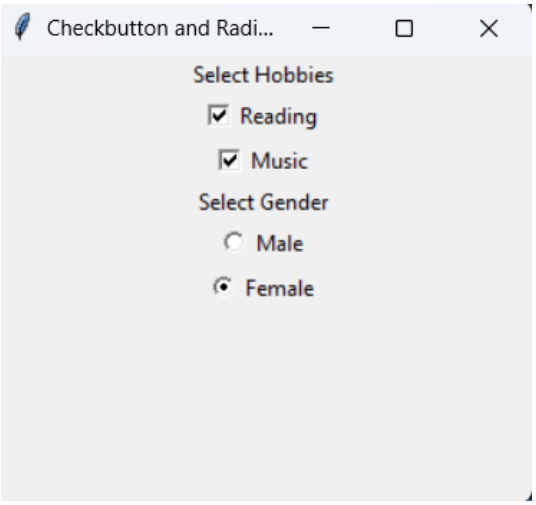
Procedure

1. Import the required module from Tkinter.
2. Create the main window using `Tk()`.
3. Add `Checkbutton` widgets for multiple selections.
4. Add `Radiobutton` widgets for single selection.
5. Use variables such as `IntVar()` or `StringVar()` to store values.
6. Arrange widgets using `pack()`.
7. Run the application using `mainloop()`.

Program

```
from tkinter import *

root = Tk()

root.title("Checkbutton and Radiobutton")

root.geometry("300x250")

c1 = IntVar()

c2 = IntVar()

r = IntVar()

Label(root, text="Select Hobbies").pack()

Checkbutton(root, text="Reading", variable=c1).pack()

Checkbutton(root, text="Music", variable=c2).pack()

Label(root, text="Select Gender").pack()

Radiobutton(root, text="Male", variable=r, value=1).pack()

Radiobutton(root, text="Female", variable=r, value=2).pack()

root.mainloop()
```

Output



Observations

- `Checkbutton` widgets allow multiple selections such as Reading and Music.
- `Radiobutton` widgets allow only one selection at a time.
- `IntVar()` is used to store the selected value of the radio button.
- When Male is selected, the value stored is `1`.
- When a Female is selected, the value stored is `2`.
- **Variable Binding Error** may occur if `variable=` is not assigned correctly with `IntVar()`.
- **Logical Error** may occur if both radio buttons are assigned the same `value`.

WEEK NO: 12

DATE: 21 March,2026

SUBJECT: Python Programming Lab

1.Question

Design and develop a GUI application using Menu and Menubutton widgets.

Procedure

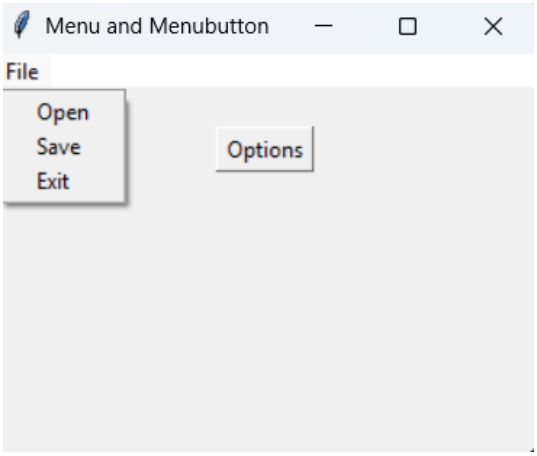
1. Import the required module from Tkinter.
2. Create the main window using `Tk()`.
3. Create a `Menu` widget and attach it to the window.
4. Add menu items such as Open and Exit.
5. Create a `Menubutton` widget.
6. Attach a dropdown menu to the menubutton.
7. Run the application using `mainloop()`.

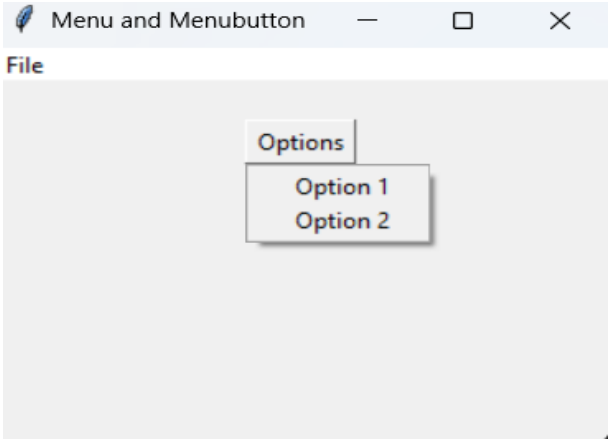
Program

```
from tkinter import *

root = Tk()

root.title("Menu and Menubutton")

root.geometry("300x200")

menu_bar = Menu(root)

root.config(menu=menu_bar)

file_menu = Menu(menu_bar, tearoff=0)

menu_bar.add_cascade(label="File", menu=file_menu)

file_menu.add_command(label="Open")

file_menu.add_command(label="Save")

file_menu.add_command(label="Exit", command=root.quit)

mb = Menubutton(root, text="Options", relief=RAISED)

mb.pack(pady=20)

mb.menu = Menu(mb, tearoff=0)
```

```
mb["menu"] = mb.menu

mb.menu.add_command(label="Option 1")

mb.menu.add_command(label="Option 2")

root.mainloop()
```

Output





Observations

- `Menu` widget creates a menu bar at the top of the window.
- `add_cascade()` is used to add menu headings.
- `add_command()` is used to add menu items.
- `Menubutton` creates a button with dropdown options.
- **Attribute Error** may occur if `mb["menu"]` is assigned incorrectly.

2.Question

Design and develop a GUI application using Listbox and Scrollbar widgets.

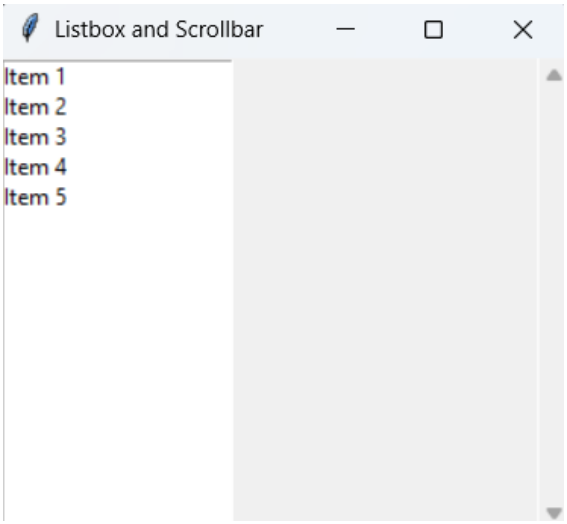
Procedure

1. Import the required module from Tkinter.
2. Create the main window using `Tk()`.
3. Create a `Scrollbar` widget.
4. Create a `Listbox` widget.
5. Link the scrollbar to the listbox using `yscrollcommand`.
6. Insert multiple items into the listbox.
7. Run the application using `mainloop()`.

program

```
from tkinter import *

root = Tk()

root.title("Listbox and Scrollbar")

root.geometry("300x250")

scroll = Scrollbar(root)

scroll.pack(side=RIGHT, fill=Y)

listbox = Listbox(root, yscrollcommand=scroll.set)

listbox.pack(side=LEFT, fill=BOTH)

for i in range(1, 6):

    listbox.insert(END, "Item " + str(i))

scroll.config(command=listbox.yview)

root.mainloop()
```

Output



Observations

- `Listbox` is used to display a list of items.
- `Scrollbar` allows scrolling through the list.
- `yscrollcommand` connects the listbox with the scrollbar.
- `insert()` is used to add items into the listbox.

3.Question

Design and develop a GUI application using Messagebox and File Dialog widget.

Procedure

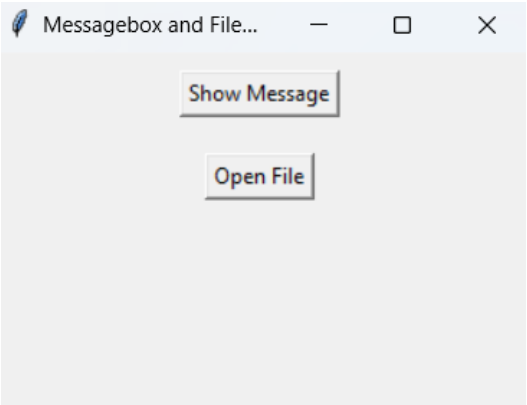
1. Import the required modules from Tkinter.
2. Create the main window using `Tk()`.
3. Add a button to display a message box.
4. Add another button to open the file dialog.
5. Use `messagebox.showinfo()` to show a popup message.
6. Use `filedialog.askopenfilename()` to open the file chooser.
7. Run the application using `mainloop()`.

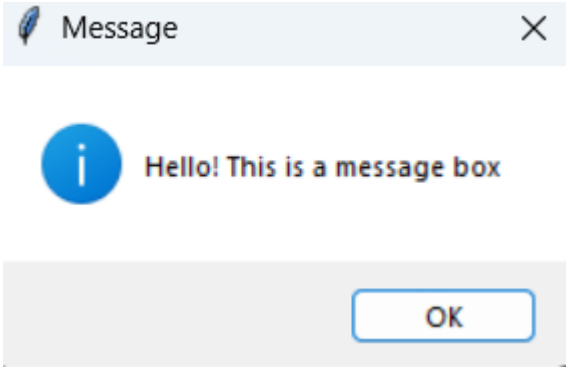
Program

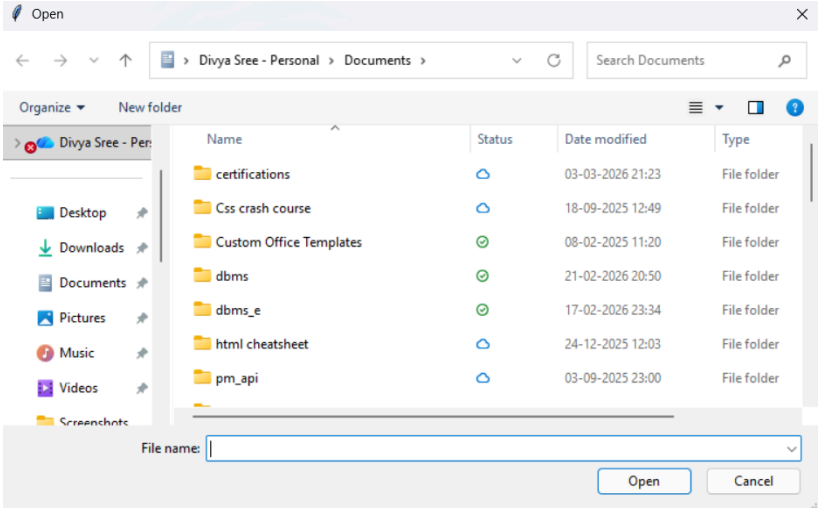
```
from tkinter import *

from tkinter import messagebox, filedialog

root = Tk()

root.title("Messagebox and File Dialog")

root.geometry("300x200")

def show_message():
```

```
    messagebox.showinfo("Message", "Hello! This is a message box")
def open_file():
    file = filedialog.askopenfilename()
    messagebox.showinfo("Selected File", file)
Button(root, text="Show Message", command=show_message).pack(pady=10)
Button(root, text="Open File", command=open_file).pack(pady=10)
root.mainloop()
```

Output







Observations

- `messagebox.showinfo()` displays a popup information message.
- `filedialog.askopenfilename()` opens a file chooser dialog box.
- Buttons trigger the corresponding functions when clicked.
- The selected file path is displayed using a message box.
- **Import Error** may occur if `messagebox` or `filedialog` is not imported properly.
- **Attribute Error** may occur if `showinfo()` or `askopenfilename()` is typed incorrectly.